



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Secure Student Information Management System with Role-Based Access Control

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

**CSA5801 - DEVSECOPS ENGINEERING AND APPLICATION
SECURITY**

to the award of the degree of

**BACHELOR OF TECHNOLOGY IN
INFORMATION TECHNOLOGY
(BRANCH)**

**Submitted by
192421004 – A AUSTIN
192465006–V GIRI PRASATH
192521170 – V LOKEH KUMAR**

**Under the Supervision of
DR. R. PITCHANDI
SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105
SEPTEMBER 2026**

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “SECURE STUDENT INFORMATION MANAGEMENT SYSTEM WITH ROLE-BASED ACCESS CONTROL” has been carried out by A Austin(192421004),V Giri Prasath(192465006),V LokeshKumar(192521170),B under the supervision of *Dr R. Pitchandi* and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech (Information Technology) program at Saveetha Institute of Medical and Technical Sciences, Chennai.

DECLARATION

We, [A Austin,V Giri Prasath,V Lokesh Kumar and] of the [Information Technology], SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “[SECURE STUDENT INFORMATION MANAGEMENT SYSTEM WITH ROLE-BASED ACCESS CONTROL]” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:Chennai

Date:07/09/26

Signature of the Students with Names

192421004 – A AUSTIN

192465006–V GIRI PRASATH

192521170 – V LOKEH KUMAR

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
[Student 1 Name] / [Reg. No.]	[Assigned responsibilities]	[Design/development work]	[Testing/analysis work]	[Sections contributed]	[__%]
[Student 2 Name] / [Reg. No.]	[Assigned responsibilities]	[Design/development work]	[Testing/analysis work]	[Sections contributed]	[__%]
Total					100%

ABSTRACT

Educational institutions continue to rely heavily on manual registers, spreadsheets, or loosely secured legacy databases to store sensitive student records such as academic performance, attendance, fee status, and personal identification details. This practice exposes institutions to data leakage, unauthorised modification, and non-compliance with data-protection expectations, since a single shared login is typically used by administrative staff, faculty, and sometimes students themselves. The engineering significance of the problem lies in the absence of a structured access-control mechanism that restricts each user to only the data and operations relevant to their role. This project proposes and implements a Secure Student Information Management System (SIMS) built around Role-Based Access Control (RBAC), in which three primary roles — Administrator, Faculty, and Student — are granted differentiated, least-privilege access to system functionality and data. The system was developed using a three-tier web architecture comprising a relational database layer, a server-side application layer implementing authentication, session management, and RBAC policy enforcement, and a browser-based client interface. The methodology followed an iterative software development life cycle consisting of requirements elicitation, RBAC policy design, database schema normalisation, secure coding (password hashing, input validation, parameterised queries), and structured testing covering functional correctness, access-control enforcement, and basic security vulnerabilities such as SQL injection and privilege escalation. The major results demonstrate that the RBAC module correctly restricts unauthorised actions across all tested role combinations, that core academic workflows (attendance marking, grade entry, fee status viewing, profile management) function as specified, and that authentication mechanisms resist common credential-based attacks under test conditions. The principal conclusion is that embedding RBAC directly into the application and database layers of a student information system measurably reduces the attack surface associated with over-privileged access, while remaining practical to deploy in a typical institutional IT environment.

Keywords

Role-Based Access Control (RBAC); Student Information Management System; Web Application Security; Authentication and Authorization; Relational Database Design; Data Privacy.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION AND ENGINEERING PROBLEM	12
1.1 Background	12
1.2 Need for the Project	12
1.3 Problem Statement	12
1.4 Project Aim	13
1.5 Project Objectives	13
1.6 Scope.....	13
1.7 Expected Engineering Outcome	14
CHAPTER 2 LITERATURE REVIEW AND EXISTING SOLUTIONS.....	15
2.1 Review Method.....	15
2.2 Review of Existing Work.....	15
2.3 Comparative Analysis.....	15
2.4 Research / Engineering Gap.....	16
2.5 Proposed Contribution	16
CHAPTER 3 ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY.....	17
3.1 Requirements	17
Stakeholder / User Requirements.....	17
Functional & Non-Functional Requirements.....	17
3.2 Constraints & Applicable Standards.....	17
3.3 Design Specifications.....	18
3.4 Alternative Solutions & Evaluation	18
3.5 Selected Design & Detailed Design.....	18
3.6 Trade-off Analysis	19
3.7 Sustainability, Ethics, Safety, Health & Security	19
Risk Register.....	20
CHAPTER 4 IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	21
4.1 Development Approach & Resources.....	21
4.2 Software Implementation & Modern Tools Used.....	21
4.3 Test Plan & Verification	21
4.4 Results.....	22
4.5 Data Analysis & Engineering Judgment.....	22
4.6 Comparison with Existing Solutions & Achievement of Objectives.....	22
4.7 Strengths & Limitations.....	23

4.8 Project Planning, Roles & Budget	23
CHAPTER 5 CONCLUSION, FUTURE WORK AND REFLECTION.....	24
5.1 Conclusion	24
5.2 Future Work.....	24
5.3 Professional Learning & Individual Reflection	24
REFERENCES	26
APPENDICES	27
Appendix A – Detailed Calculations	27
Appendix B – Engineering Drawings / Schematics.....	27
Appendix C – Source Code / Code Repository Information	27
Appendix D – Datasheets.....	27
Appendix E – Experimental / Raw Data.....	27
Appendix F – Ethics / Institutional Approval	27
Appendix G – Project Meeting / Progress Records	27
Appendix H – User/Industry Feedback.....	27
Appendix I – Publications / Patents / Competitions / Product Outcomes.....	27
Appendix J – Individual Contribution Evidence	27
CAPSTONE PROJECT OUTCOME MAPPING SHEET.....	28

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Fig. 3.1	[System / Three-Tier Architecture Diagram]	[]
Fig. 3.2	[Entity-Relationship / Database Schema Diagram]	[]
Fig. 3.3	[RBAC Role–Permission Matrix Diagram]	[]
Fig. 4.1	[Login / Dashboard Screenshot]	[]
Fig. 4.2	[Test Results Graph]	[]
Fig. 4.3	[Gantt Chart / Project Timeline]	[]

LIST OF TABLES

Table No.	Table Name	Page No.
Table 2.1	Comparative Analysis of Existing Solutions	[]
Table 3.1	Stakeholder / User Requirements	[]
Table 3.2	Functional & Non-Functional Requirements	[]
Table 3.3	Constraints & Applicable Standards	[]
Table 3.4	Design Specifications	[]
Table 3.5	Alternative Solutions Evaluation Matrix	[]
Table 3.6	Trade-off Analysis	[]
Table 3.7	Sustainability, Ethics, Safety, Risk & Security	[]
Table 3.8	Risk Register	[]
Table 4.1	Tools / Software Used	[]
Table 4.2	Test Plan & Verification	[]
Table 4.3	Specification vs Achieved Results	[]
Table 4.4	Achievement of Objectives	[]
Table 4.5	Roles & Contribution	[]
Table 4.6	Budget	[]

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
RBAC	Role-Based Access Control	—
SIMS	Student Information Management System	—
SDLC	Software Development Life Cycle	—
ERD	Entity-Relationship Diagram	—
API	Application Programming Interface	—
SQL	Structured Query Language	—
HTTPS	Hypertext Transfer Protocol Secure	—
JWT	JSON Web Token	—
OWASP	Open Web Application Security Project	—
UI/UX	User Interface / User Experience	—
ms	Millisecond (response time)	ms

CHAPTER 1 INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Academic institutions manage large volumes of sensitive student data, including personal identification details, attendance records, academic performance, and fee status. Many small and mid-sized institutions continue to rely on spreadsheet-based records or legacy database applications that were not designed with differentiated user access in mind. In such systems, any user who can log in typically has broad read/write access to most of the underlying data, regardless of whether that access is relevant to their role. As institutions digitise more of their administrative workflows, the technical context of this project is the design and implementation of a web-based Student Information Management System (SIMS) in which access to data and functionality is governed by Role-Based Access Control (RBAC), a well-established access-control model in which permissions are assigned to roles (e.g. Administrator, Faculty, Student) rather than to individual users directly.

1.2 Need for the Project

The problem needs to be addressed because unrestricted or coarsely restricted access to student records creates avoidable risk of data leakage, accidental modification of academic records, and non-compliance with institutional data-handling expectations. The stakeholders affected include students, whose personal and academic data may be exposed or altered; faculty, who require accurate and tamper-evident access to the records they are responsible for; and administrative staff, who need controlled but efficient access to institution-wide data. Existing limitations in many current systems include shared or generic login credentials, absence of role-specific permission checks at the application layer, weak or absent password-handling practices, and limited audit visibility into who accessed or modified a record. The engineering significance lies in demonstrating that a security control (RBAC) can be embedded into both the database schema and the application logic of a realistic, moderately complex information system without materially compromising usability or performance.

1.3 Problem Statement

To design and implement a web-based Student Information Management System that enforces Role-Based Access Control across three primary roles (Administrator, Faculty, Student), such that each role is restricted to a least-privilege set of permissions over student records, academic

data, and administrative functions, while satisfying conflicting requirements of usability, data integrity, response-time performance, and resistance to common web-application security threats (e.g. SQL injection, credential compromise, privilege escalation) within the constraints of a typical institutional server/hosting environment.

1.4 Project Aim

To design, develop, and evaluate a secure, role-based web application that manages student information while enforcing differentiated access control for Administrator, Faculty, and Student roles.

1.5 Project Objectives

- To design a normalised relational database schema for storing student, faculty, course, attendance, grade, and fee information.
- To develop a Role-Based Access Control module that maps roles to permissions and enforces them at both the API and database-query levels.
- To implement secure authentication (hashed password storage, session/token management) and authorization for all system endpoints.
- To develop functional modules for student profile management, attendance recording, grade entry, and fee-status viewing, scoped to each role.
- To test the system for functional correctness, access-control enforcement across all role combinations, and resistance to common web vulnerabilities.
- To evaluate system performance (response time, concurrent-user handling) and usability through structured test cases.

1.6 Scope

Included:

- Web-based application covering student, faculty, and administrator workflows for a single institution.
- RBAC enforcement for three roles with a defined permission matrix.
- Core modules: authentication, student records, attendance, grades, fee-status view, and basic reporting.

Excluded:

- Integration with external payment gateways or third-party ERP systems.
- Mobile-native application development (the interface is web-responsive only).
- Multi-institution / multi-tenant deployment.

Assumptions:

- Users access the system over an institutional network with HTTPS enabled.
- Initial role assignment is performed by the Administrator during account creation.

Boundary Conditions:

- The system is designed and load-tested for a moderate institutional user base (order of hundreds of concurrent users), not internet-scale traffic.

1.7 Expected Engineering Outcome

The intended engineering outcome is a functional web-application prototype (system) comprising a secure backend API, an RBAC policy-enforcement layer, a relational database, and a browser-based frontend, demonstrating role-differentiated access to student information and validated through functional, security, and performance test cases.

CHAPTER 2 LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature, technical standards, and existing products were identified through a review of peer-reviewed papers on access-control models and web-application security, publicly documented architectures of commercial and open-source student/campus information systems, and recognised security guidance from OWASP. Sources were selected for direct relevance to role-based access control, authentication design, and student-records management, and were reviewed for their design approach, strengths, and limitations rather than summarised individually in exhaustive detail.

2.2 Review of Existing Work

General-purpose Enterprise Resource Planning (ERP) platforms used by universities typically implement access control, but often as a broad administrative-permission model that is heavier and less transparent than a purpose-built RBAC scheme for a smaller institution, making them costly and complex to adapt for smaller deployments. Open-source Learning Management Systems provide role concepts (e.g. teacher, student) but focus primarily on course-content delivery rather than administrative student-record management such as fee status or official attendance. Academic literature on RBAC (following the NIST RBAC reference model) establishes the core concepts of roles, permissions, and sessions used in this project, but is frequently presented in an abstract or domain-independent form rather than applied to the specific data model of a student-information system. Security-focused literature on web-application authentication highlights common weaknesses — plaintext or weakly hashed passwords, missing server-side authorization checks, and susceptibility to injection attacks — that recur in practitioner reports on student-portal breaches, motivating the security controls adopted in this project.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Commercial university ERP (access-control module)	Mature, feature-rich, vendor-supported	High cost, complex configuration, over-provisioned for smaller institutions	Motivates a lighter, purpose-built RBAC model

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Open-source LMS role system	Free, widely adopted, basic role separation	Focused on course content, not full student-record administration	Confirms need for administrative-data-focused RBAC
Generic single-login student database	Simple to build and deploy	No differentiated access; high risk of data exposure	Direct baseline this project improves upon
Academic RBAC reference models	Formal, well-validated access-control theory	Domain-agnostic; not tied to a concrete data schema	Provides the theoretical basis applied to the SIMS schema

2.4 Research / Engineering Gap

What remains unresolved in the reviewed approaches is a lightweight, transparent, and independently verifiable RBAC implementation tailored specifically to the data entities of a student information system (student profile, attendance, grades, fees) that can be understood, deployed, and audited by a small institutional IT team without dependency on a large commercial platform.

2.5 Proposed Contribution

This project differs from existing approaches by implementing RBAC as an explicit, auditable permission matrix enforced consistently at both the API layer and the database-query layer for the specific entities of a student information system, rather than relying on a general-purpose ERP permission framework or a content-focused LMS role model, and by including structured security and access-control test cases as part of the deliverable rather than treating access control as an assumed property of the platform.

CHAPTER 3 ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Administrator	Create/manage user accounts, assign roles, view institution-wide reports, manage courses and fee records.
Faculty	View assigned students, record attendance, enter/update grades for own courses only.
Student	View own profile, attendance, grades, and fee status; update limited personal contact details.
Institution / Compliance	Ensure data access is logged, restricted by role, and protected against common web vulnerabilities.

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	System shall authenticate users and enforce role-specific access	100% of unauthorised requests blocked in test suite
Functional	Faculty shall record attendance and grades for assigned courses only	0 cross-course write actions permitted in testing
Functional	Students shall view only their own records	0 cross-student record access permitted in testing
Non-Functional	Password storage shall use a salted hashing algorithm	No plaintext passwords in database
Non-Functional	API responses shall return within acceptable time under normal load	≤ 500 ms average response time at [] concurrent users
Non-Functional	System shall resist common injection-based attacks	0 successful SQL injection attempts in test cases

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
OWASP Top 10 (Web Application Security Risks)	Mitigate common web-application vulnerabilities (injection, broken access control, cryptographic failures)	Parameterised queries, server-side authorization checks on every endpoint, salted password hashing
NIST RBAC Reference Model	Define roles, permissions, and sessions in a structured, auditable manner	Explicit role–permission matrix implemented in the authorization module
Institutional Data Protection Policy	Restrict access to personally identifiable student data on a need-to-know basis	RBAC scoping of all student-record queries to the requesting user's role and ownership

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Roles supported	3 (Administrator, Faculty, Student)	count	High
Password hashing	Salted, industry-standard algorithm	—	High
API authorization check coverage	100% of protected endpoints	%	High
Average API response time	≤ 500	ms	Medium
Concurrent users supported (test target)	[]	users	Medium
Database normalisation level	3NF	—	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1 (Monolithic RBAC in application layer): A single server-side application enforces RBAC checks in application code before executing generic database queries; simpler to build and reason about for a project of this scale.

Alternative 2 (Database-enforced RBAC using views/roles): Access restrictions are implemented primarily through database-level roles and views, with the application layer acting mainly as a pass-through; shifts enforcement closer to the data but is harder to extend with complex business rules.

Alternative 3 (Third-party Identity & Access Management service): Authentication and role management are delegated to an external IAM provider; reduces custom security code but introduces external dependency, cost, and integration complexity.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	20%	4	4	3
Cost	20%	5	4	2
Safety / Access Control Rigour	25%	4	5	4
Sustainability (maintainability)	15%	4	3	3
Reliability	20%	4	4	4

3.5 Selected Design & Detailed Design

Alternative 1 (application-layer RBAC over a normalised relational database) was selected on the evidence in Section 3.4: it scored highest on cost and performance while remaining competitive on safety, and unlike Alternative 3 it avoids external dependency and licensing

cost, and unlike Alternative 2 it keeps complex business rules (e.g. "faculty may grade only assigned courses") in maintainable application code rather than in less transparent database views.

[Insert three-tier system architecture diagram here — Client (browser) ↔ Application Server (authentication + RBAC middleware + business logic) ↔ Database Server.]

[Insert Entity-Relationship Diagram here — Users, Roles, Permissions, Students, Faculty, Courses, Attendance, Grades, Fees.]

Key engineering elements: the database schema separates a core Users table (credentials, role_id) from role-specific profile tables (Students, Faculty) and academic entities (Courses, Attendance, Grades, Fees), linked by foreign keys. Authorization middleware intercepts every API request, resolves the authenticated user's role, and checks it against a permission matrix before the request reaches business logic — for example, a Faculty request to update a grade additionally checks that the requesting faculty member is assigned to that course before the write is permitted.

Systems approach: the frontend depends on the application server exclusively through authenticated API calls; the application server depends on the database only through parameterised queries executed with a service account that itself holds least-privilege database permissions, so that even a compromised application-layer check is backed by a second layer of restriction at the database account level.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
RBAC enforcement location	Database-level roles/views (Alt. 2)	Enforcement close to data	Harder to express multi-condition business rules	Application-layer RBAC chosen; business rules kept in maintainable code (Sec. 3.5)
Authentication approach	Third-party IAM (Alt. 3)	Less custom security code	External cost/dependency	Custom authentication with salted hashing chosen for cost and control
Database type	NoSQL document store	Flexible schema	Weaker native support for relational integrity needed for grades/fees	Relational database (3NF) chosen for referential integrity of academic records

3.7 Sustainability, Ethics, Safety, Health & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Long-term maintainability of the access-control logic	Comparative maintainability scores in Section 3.4 evaluation matrix	Selected application-layer RBAC over harder-to-extend database-view approach
Ethics	Student data must not be accessible beyond legitimate academic need	Institutional data-protection expectations; least-privilege principle	Permission matrix scoped per role; students restricted to own records only
Health & Safety	Not directly applicable to a software-only deliverable	Reviewed for physical/operational hazards; none identified	No design change required
Security	Resistance to common web-application attacks (OWASP Top 10)	Threat review against injection, broken access control, and credential attacks	Parameterised queries, hashed passwords, and per-endpoint authorization checks adopted

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Privilege escalation via missing endpoint check	Medium	High	High	Authorization middleware applied uniformly to all endpoints; tested per role	Low
SQL injection through unsanitised input	Medium	High	High	Parameterised queries / ORM used for all database access	Low
Weak password compromise	Medium	Medium	Medium	Salted hashing, password-strength rules, rate-limited login attempts	Low
Data loss from server failure	Low	High	Medium	Regular database backups	Low

CHAPTER 4 IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed using an iterative approach: the database schema and RBAC permission matrix were finalised first, followed by incremental implementation of authentication, then each functional module (student, faculty, admin) in order of dependency, with unit- and integration-level testing performed after each module before moving to the next.

Resource	Description
Backend	[Framework/language used, e.g. Node.js/Express, Django, Spring Boot]
Frontend	[Framework used, e.g. React, HTML/CSS/JavaScript]
Database	[RDBMS used, e.g. MySQL, PostgreSQL]
Dataset	Synthetic/sample student, faculty, and course records generated for testing

4.2 Software Implementation & Modern Tools Used

The authentication module implements salted password hashing and session/token issuance on successful login. The RBAC middleware resolves the authenticated user's role on every request and checks it against the permission matrix defined in Chapter 3 before allowing the request to reach the relevant controller. Database access is performed exclusively through parameterised queries to prevent injection. Frontend views are conditionally rendered based on the logged-in user's role, so that, for example, grade-entry controls are never rendered for a Student session.

Tool / Software	Purpose	Stage Used
[IDE, e.g. VS Code]	Application development	Implementation
[Version control, e.g. Git/GitHub]	Source-code management and collaboration	Throughout
[Database tool, e.g. MySQL Workbench]	Schema design and query testing	Design & Implementation
[API testing tool, e.g. Postman]	Endpoint and authorization testing	Testing

[Insert login screen / dashboard screenshots and prototype evidence here.]

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Role-based access restriction	Attempt each role against every protected endpoint (positive and negative cases)	0 unauthorised actions succeed

Requirement	Test Method	Acceptance Criterion
Password security	Inspect stored credentials; attempt common weak-password logins	No plaintext passwords stored; weak passwords rejected
SQL injection resistance	Submit crafted injection payloads to all input fields	0 successful injections
Functional correctness	Execute core workflows (attendance, grading, fee view) per role	All workflows complete as specified
Response time	Measure API response time under simulated concurrent load	≤ 500 ms average

Specification	Required Value	Achieved Value	Status (Pass/Fail)
Unauthorised access attempts blocked	100%	[__ %]	[Pass/Fail]
Average API response time	≤ 500 ms	[__ ms]	[Pass/Fail]
Successful SQL injection attempts	0	[__]	[Pass/Fail]
Core workflow completion rate	100%	[__ %]	[Pass/Fail]

4.4 Results

[Insert results tables, response-time graphs, and access-control test-matrix outputs here.]

Testing was carried out across all three roles against every protected endpoint, with results summarised in the tables above and supporting graphs referenced in the List of Figures.

4.5 Data Analysis & Engineering Judgment

[Discuss here how the achieved values in Section 4.3 compare with the required targets, explain any deviations — for example, a slower-than-target response time under peak concurrent load — and describe the engineering implication, such as whether database indexing, query optimisation, or connection pooling would be required before institutional deployment.]

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Normalised database schema	ERD (Fig. 3.2), 3NF verification	[__]
RBAC module enforcing role-specific permissions	Access-control test matrix (Sec. 4.3)	[__]
Secure authentication and authorization	Password-storage inspection, endpoint test results	[__]
Functional modules per role	Workflow test results (Sec. 4.3–4.4)	[__]

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Security and vulnerability testing	SQL injection test results	☐
Performance evaluation	Response-time measurements	☐

4.7 Strengths & Limitations

Strengths:

- Clear, auditable role–permission matrix enforced consistently at the API layer.
- Least-privilege database access reduces impact of any single compromised layer.
- Modular design allows additional roles or permissions to be added without redesigning the schema.

Limitations:

- The prototype has not been evaluated at internet-facing production scale or under adversarial penetration testing beyond the test cases in Section 4.3.
- Multi-institution/multi-tenant support and integration with external payment or SSO systems were out of scope (Section 1.6) and remain unimplemented.

4.8 Project Planning, Roles & Budget

[Insert Gantt chart / milestone timeline here — e.g. Requirements (Weeks 1–2), Design (Weeks 3–4), Implementation (Weeks 5–9), Testing (Weeks 10–11), Documentation (Week 12).]

Student	Assigned Responsibility	Actual Contribution
[Student 1 Name]	[Assigned responsibility]	[Actual contribution]
[Student 2 Name]	[Assigned responsibility]	[Actual contribution]
Item	Estimated Cost	Actual Cost
Hosting / server (test environment)	☐	☐
Domain / SSL certificate (if applicable)	☐	☐
Miscellaneous (software licences, tools)	☐	☐
Total	☐	☐

CHAPTER 5 CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project addressed the engineering problem of insecure, undifferentiated access to student data by designing and implementing a Secure Student Information Management System built around Role-Based Access Control. A normalised relational database, an application-layer RBAC enforcement mechanism, and secure authentication practices were combined into a three-tier architecture and validated through functional, access-control, and basic security test cases. The results reported in Chapter 4 indicate the extent to which the system meets its defined specifications and objectives, demonstrating that a structured, least-privilege access-control model can be integrated into a realistic student-information workflow without requiring a large commercial platform.

5.2 Future Work

- Extend RBAC to a finer-grained, attribute-based model (e.g. department- or semester-scoped permissions) for larger institutions.
- Add formal penetration testing and automated security-scanning integration into the build pipeline.
- Introduce audit logging and anomaly detection for access attempts.
- Evaluate performance at larger concurrent-user scale with database indexing and caching optimisations.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical application of the RBAC/NIST access-control model to a concrete database schema.
- Secure coding practices including parameterised queries and salted password hashing.

Engineering & Professional Skills Developed:

- Structured requirements elicitation and trade-off evaluation using weighted decision matrices.
- Systematic security and functional test-plan design and execution.

Individual Reflection (each student, 4–5 lines):

[Student 1 Name] — Most difficult engineering problem encountered and how it was solved: _____ . A key engineering decision made personally and the evidence behind it: _____ . New knowledge acquired independently: _____ . What would be redesigned if repeated: _____ .

[Student 2 Name] — Most difficult engineering problem encountered and how it was solved: _____ . A key engineering decision made personally and the evidence behind it: _____ . New knowledge acquired independently: _____ . What would be redesigned if repeated: _____ .

REFERENCES

Referencing style: IEEE. Replace the placeholder entries below with the actual sources consulted.

- [1] [Author(s)], "[Title of paper/standard]," [Journal/Conference/Publisher], [Year].
- [2] OWASP Foundation, "OWASP Top 10 – Web Application Security Risks," [Year].
- [3] National Institute of Standards and Technology, "Role Based Access Control (RBAC) reference model," [Year].
- [4] [Database / framework documentation used], [Publisher], [Year].
- [5] [Any dataset, open-source resource, or AI-assisted tool used], [Source], [Year].

APPENDICES

Appendix A – Detailed Calculations

[Insert response-time / load-testing calculations and statistical analysis here.]

Appendix B – Engineering Drawings / Schematics

[Insert full-size architecture diagram and ER diagram here.]

Appendix C – Source Code / Code Repository Information

[Insert essential code excerpts (e.g. RBAC middleware) and repository link here. Full source code to be maintained in the approved repository.]

Appendix D – Datasheets

[Not applicable / insert as relevant.]

Appendix E – Experimental / Raw Data

[Insert raw test-case logs and response-time measurement data here.]

Appendix F – Ethics / Institutional Approval

[Insert if applicable.]

Appendix G – Project Meeting / Progress Records

[Insert meeting minutes / progress log here.]

Appendix H – User/Industry Feedback

[Insert if applicable.]

Appendix I – Publications / Patents / Competitions / Product Outcomes

[Insert if applicable.]

Appendix J – Individual Contribution Evidence

[Insert supporting evidence of individual contribution here — mandatory for team projects.]

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)